

Securing MySQL Server for Baldwin County Health Department

Hayden Sapp

Department of Cyber Intelligence and Security

CI340: Database Security

Professor Reid

April 17, 2024

Introduction

The Baldwin County Health Department needs a MySQL service and workbench for Frank, Adin, and Pamela to ultimately provide insightful reports to local clinics. As the bastion of public health services in Alabama, they will be using Ubuntu 22.04, an open-source Linux distribution to keep costs low and ensure that the reports generated by the Covid19 data will be available to access for each local clinic no matter the operating system. This system also needs to remain secure from potential attack, and as such it is important that not only the MySQL service is secure, but the operating system is as well. The local users and MySQL users on this system will abide by the principle of least privilege and only have access to the objects they need access to.

Operating System Security

Environment Setup

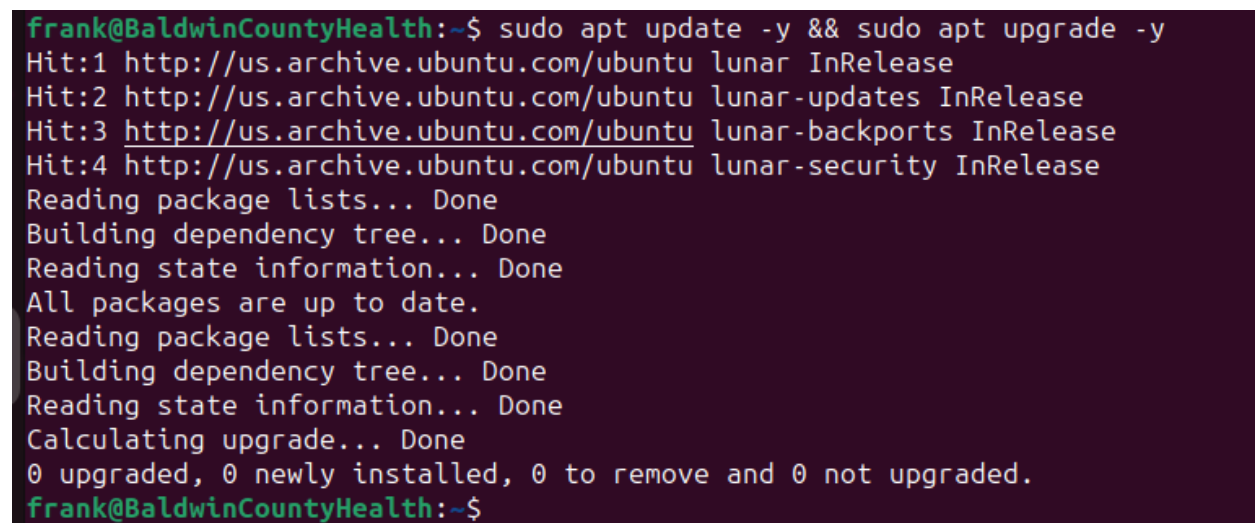
For this lab, the operating system used is Ubuntu 22.04, the latest stable release from Canonical available from their website. The machine's name is *BaldinCountyHealth* and it currently has three local users: Adin, Frank, and Pamela—who's login information is shown below. Additionally, the root password has been changed from its default.

- frank password: TastyPotatoes1234@
- adin password: Ar1zon@Tea11
- pamela password: OcypodidaeQuadrata
- root: UrzasTower1234

System and Package Updates

Before any installation of MySQL and any user operation, the operating system and its package repository needs to be up to date. Figure 1 shows that the repository list has been updated with the *apt update* command and then the updates are installed with the *apt upgrade* command.

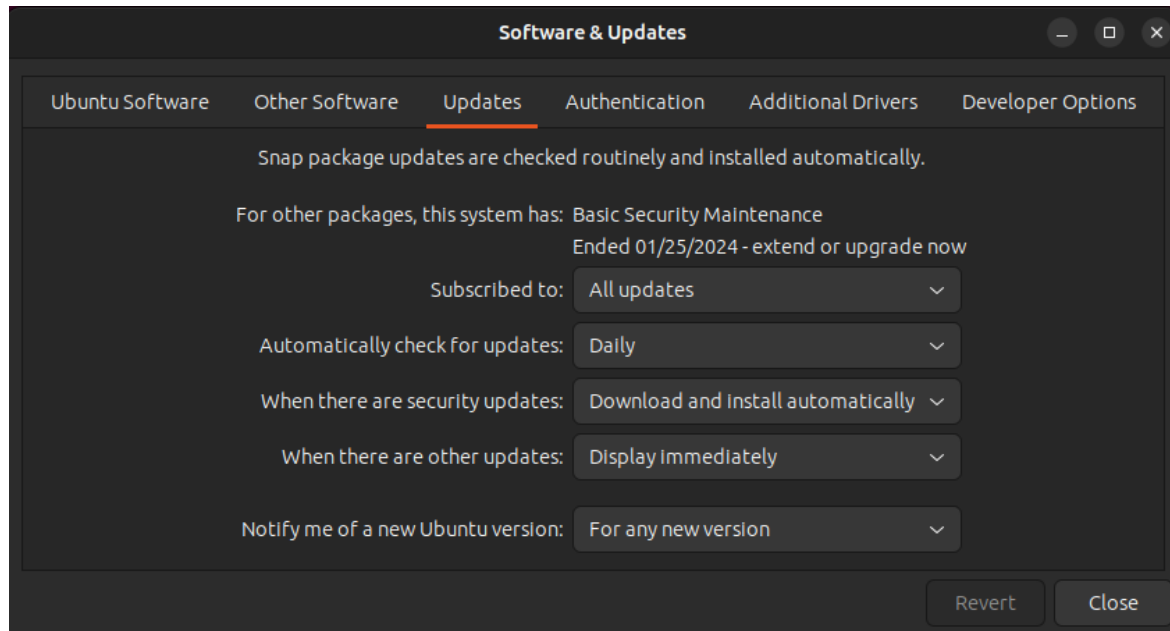
Figure 1



```
frank@BaldwinCountyHealth:~$ sudo apt update -y && sudo apt upgrade -y
Hit:1 http://us.archive.ubuntu.com/ubuntu lunar InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu lunar-updates InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu lunar-backports InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu lunar-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
frank@BaldwinCountyHealth:~$
```

Note. This figure shows the output of the command “*sudo apt update -y && sudo apt upgrade -y*” which displays the output that there have been 0 packages upgraded and 0 newly installed, showing that the system and the installed packages are up to date.

Following the installation of updates, settings need to be changed in order to have Ubuntu automatically check for updates and then install them (if available). Figure 2 shows the settings page where this information can be changed, with the new update settings being set to automatically check for updates daily, automatically install updates immediately, and always display the user when there is a new update available.

Figure 2

Note: This figure shows the *Software and Updates* settings which were changed from their system defaults. The notable change in this environment is the “*Download and Install automatically*” and “*Automatically check for updates*” set to daily.

Installing MySQL Server and MySQL Workbench

MySQL Server is the package desired for Pamela and her team to be able to read, access, and modify the health data. As it is part of the apt repository list by default, it can be installed from the command line with no additional steps, as shown in figure 3. Additionally, with the download coming from the apt repository, Ubuntu has ensured that it is the latest version of the package, and it is checked with the stored checksum to ensure its integrity through download.

Figure 3

```

frank@BaldwinCountyHealth:~/Desktop$ sudo apt install mysql-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mysql-server is already the newest version (8.0.35-0ubuntu0.23.04.1).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
frank@BaldwinCountyHealth:~/Desktop$

```

Note. This figure shows the command “*sudo apt install mysql-server*” which shows that the package has already been installed. While not being natively installed in Ubuntu 22.04, this output was significantly shorter than the actual installation and thus included in this report.

After installing MySQL Server, MySQL Workbench must be installed for users to easily access the local database. Unlike the server, MySQL Workbench is not currently in the apt list and has to be added by installing the package from <https://dev.mysql.com/downloads/repo/apt/>. Following the download, the Debian file need to be depackaged and added to the apt repository list with the following command: “*sudo dpkg -i mysql-apt-config_0.8.29-1_all.deb*”. Following the unpacking of the package, the apt repository list needs to be updated once more and then MySQL Workbench can be installed, shown in figure 4.

Figure 4

```

frank@BaldwinCountyHealth:~/Downloads$ sudo apt install mysql-workbench-community
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mysql-workbench-community is already the newest version (8.0.33-1ubuntu23.04).
The following packages were automatically installed and are no longer required:
  libcgi-fast-perl libcgi-pm-perl libevent-core-2.1-7 libevent-pthreads-2.1-7 libfcgi-bin libfcgi-perl
  libfcgi0ldbl libhtml-template-perl libprotobuf-lite32
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
frank@BaldwinCountyHealth:~/Downloads$

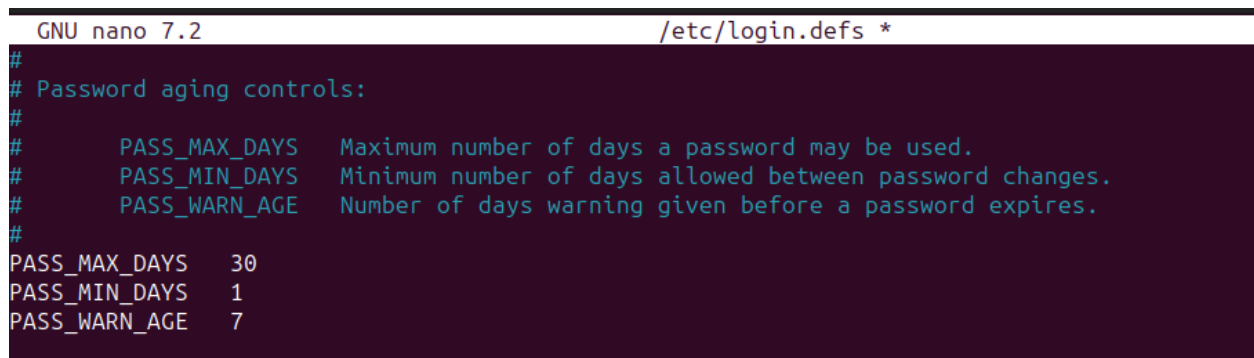
```

Note. This figure shows that MySQL Workbench has been installed by the apt repository.

Password Policy Security

In order to keep the local users on the system secure, the password policies need to be updated from their defaults—as the majority of the default settings are set to null. The primary password policy file is *login.defs*, which controls the user login control. The password policies chosen for this environment is shown in figure 5. The passwords need to be changed every 30 days, the passwords can only be changed once a day, and the user gets warned a week before their password expires. Additionally, the last 5 passwords will be remembered to ensure that the user does not cycle through 2 passwords on the system, shown in figure 6.

Figure 5



```
GNU nano 7.2 /etc/login.defs *
#
# Password aging controls:
#
#     PASS_MAX_DAYS   Maximum number of days a password may be used.
#     PASS_MIN_DAYS   Minimum number of days allowed between password changes.
#     PASS_WARN_AGE   Number of days warning given before a password expires.
#
PASS_MAX_DAYS   30
PASS_MIN_DAYS    1
PASS_WARN_AGE    7
```

Note. This figure shows the changed values from *login.defs* to fit the chosen password policy.

Figure 6

```

GNU nano 7.2 /etc/pam.d/common-password *
#11. Without this option, the default is Unix crypt. Prior releases
#used the option "sha512"; if a shadow password hash will be shared
#between Debian 11 and older releases replace "yescrypt" with "sha512"
#for compatibility. The "obscure" option replaces the old
#`OBSOLETE_CHECKS_ENAB' option in login.defs. See the pam_unix manpage
#for other options.

# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules. See
# pam-auth-update(8) for details.

# here are the per-package modules (the "Primary" block)
password      requisite          pam_pwquality.so retry=3
password      [success=2 default=ignore] pam_unix.so obscure use_authtok try_first_pass yescrypt remember=5

```

Note. This figure shows the added line of remember=5 in the pam.d configuration files.

Account Policy Security

As there should only be 3 local users on the computer, Frank has been given administrative access to the computer as the cyber security specialist. Pamela and Adin do not have access to administrative features or the sudo command. As no one else should have access to the system, the guest account has been disabled (figure 7), and the root account has been locked (figure 8). Additionally, the root account has had its password changed from its default in the event the account is unlocked (figure 9).

Figure 7

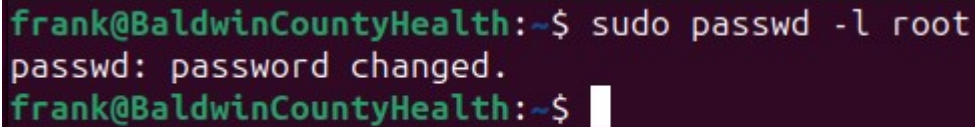
```

GNU nano 7.2 /usr/share/lightdm/lightdm.conf.d/50-ubuntu.conf *
[Seat:*]
user-session=ubuntu
allow-guest=false

```

Note. This figure shows that the guest account is disabled when asked for login credentials.

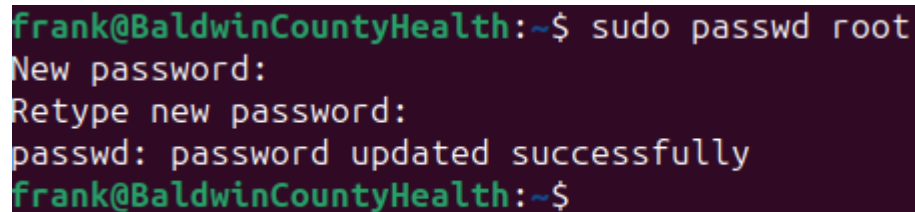
Figure 8



```
frank@BaldwinCountyHealth:~$ sudo passwd -l root
passwd: password changed.
frank@BaldwinCountyHealth:~$
```

Note. This figure shows that the root account has been locked with the “*passwd -l*” command.

Figure 9



```
frank@BaldwinCountyHealth:~$ sudo passwd root
New password:
Retype new password:
passwd: password updated successfully
frank@BaldwinCountyHealth:~$
```

Note. This figure shows that the root account’s password has been changed from its default.

Additional Security Features

Additional security mechanisms have been added to the operating system, as shown in figure 10. From this configuration, the operating system will ignore any ping broadcasts to prevent ping flood Denial of Service (DOS) attacks. Additionally, this configuration prevents TCP DOS attacks by remembering SYN cookies. To prevent known buffer overflow attacks, the kernel will randomize the memory addresses it starts with when loading a program into active memory.

Figure 10


```

GNU nano 7.2 /etc/sysctl.conf *
#net.ipv4.conf.all.send_redirects = 0
#
# Do not accept IP source route packets (we are not a router)
#net.ipv4.conf.all.accept_source_route = 0
#net.ipv6.conf.all.accept_source_route = 0
#
# Log Martian Packets
#net.ipv4.conf.all.log_martians = 1
#
#####
# Magic system request Key
# 0=disable, 1=enable all, >1 bitmask of sysrq functions
# See https://www.kernel.org/doc/html/latest/admin-guide/sysrq.html
# for what other values do
#kernel.sysrq=438

net.ipv4.icmp_echo_ignore_broadcasts=1
net.ipv4.tcp_syncookies=1
kernel.randomize_va_space=2

```

Note. This figure shows the ignorance of ping broadcasts, the remembrance of SYN cookies, and the randomization of memory.

The firewall also must be turned on with additional settings to meet the MySQL needs.

Figure 11

```

frank@BaldwinCountyHealth:~$ sudo ufw enable
Firewall is active and enabled on system startup

```

Note. This figure just shows that the firewall is on and active during startup.

Figure 12

```

adin@BaldwinCountyHealth:~$ sudo mv
[sudo] password for adin:
adin is not in the sudoers file.

```

Note. This figure shows that the user account access control works, given that Adin does not have the administrative access that Frank does.

SQL Setup

For this SQL installation, each main user will have their own local account with the connection title being their role. Frank's account has the connection "Cyber Security", Adin's account has the connection "Database Administrator", and Pamela uses the "Data Researcher" connection. This is done to mimic role-based access control, giving permissions to the roles rather than the users themselves. While role-based access control can't be implemented into MySQL, this serves as a template for others to replicate—as if a data researcher other than Pamela needed access, they could request an account be made and then copy the same connection settings as Pamela. The Account login information is shown below:

- frank password: Brachyura!23
- pamela password: Xantidae!!
- adin password: SecurityProfessional1

Each account has been made with a max number of connections and concurrent connections set to 5 and 1 respectively, shown in figure 13. As Frank has permissions for the Process Admin, he needs 2 concurrent connections rather than one, so that MySQL can report the process information. Limiting the connections made per hour and the number of concurrent connections from users helps mitigate DOS attacks against the MySQL service.

Figure 13

Login	Account Limits	Administrative Roles	Schema Privileges
Max. Queries:	0	Number of queries the account can execute within one hour.	
Max. Updates:	0	Number of updates the account can execute within one hour.	
Max. Connections:	5	The number of times the account can connect to the server per hour.	
Concurrent Connections:	1	The number of simultaneous connections to the server the account can have.	

Note. This figure shows the generalized account settings for max and concurrent connections.

As Frank is the Cyber Security Specialist of the Baldwin County Health Department, he has been given the roles of Maintenance Administrator, Process Administrator, User Administrator, and Security Administrator, shown in figure 13. This ensures that he can perform his duties in hardening the MySQL service. Notably, Frank cannot view, add, or delete data in the databases. As a data researcher, Pamela should only have permission to view and index any data on the schema's she has access to. It is imperative that data researchers do not have the ability to modify the data that they can view. As the Database Administrator, Adin's permissions are set to be the Database Manager, Database Designer, and Backup Administrator to create schemas, load data, and make backups.

Figure 14

Role	Description
☐ DBA	grants the rights to perform all tasks
☒ MaintenanceAdmin	grants rights needed to maintain server
☒ ProcessAdmin	rights needed to assess, monitor, and kill any user process running in server
☒ UserAdmin	grants rights to create users logins and reset passwords
☒ SecurityAdmin	rights to manage logins and grant and revoke server and database level permission
☐ MonitorAdmin	minimum set of rights needed to monitor server
☐ DBManager	grants full rights on all databases
☐ DBDesigner	rights to create and reverse engineer any database schema
☐ ReplicationAdmin	rights needed to setup and manage replication
☐ BackupAdmin	minimal rights needed to backup any database

Note. This figure shows Frank's permissions and roles in the MySQL Service.

Figure 15

☒ INDEX
☐ INSERT
☐ LOCK TABLES
☐ PROCESS
☐ REFERENCES
☐ RELOAD
☐ REPLICATION CLIENT
☐ REPLICATION SLAVE
☒ SELECT

Note. This figure shows Pamela's permissions are only to select and index the information in the database.

Figure 16

Login	Account Limits	Administrative Roles	Schema Privileges
Role		Description	
☐	DBA	grants the rights to perform all tasks	
☐	MaintenanceAdmin	grants rights needed to maintain server	
☐	ProcessAdmin	rights needed to assess, monitor, and kill any user process running in server	
☐	UserAdmin	grants rights to create users logins and reset passwords	
☐	SecurityAdmin	rights to manage logins and grant and revoke server and database level permission	
☐	MonitorAdmin	minimum set of rights needed to monitor server	
☒	DBManager	grants full rights on all databases	
☒	DBDesigner	rights to create and reverse engineer any database schema	
☐	ReplicationAdmin	rights needed to setup and manage replication	
☒	BackupAdmin	minimal rights needed to backup any database	
☐	Custom	custom role	

Note. This figure shows Adin's permissions as the Database manager.

Auditing and Security Settings

In this instance of MySQL, error logs, general logs, and slow query logs have been enabled and saved on the system, shown in figure 17. This ensures that all the processes are logged on the system and are able to be monitored by the database administrator. All of these logs are stored in `/var/log/mysql` to have one destination for every log that MySQL will generate. Additionally, the max number of connections, max number of users, and connection timeout have all been set to mitigate DOS attacks. The default port for MySQL has also been changed from 3306 to 1781 to reduce the noticeability of the MySQL service from attackers. As the port is changed from default, the firewall needs to allow port 1781 to allow connections to the MySQL instance from the workbench, shown in figure 18.

Figure 17

```
[mysqld]
innodb_force_recovery = 1
log_error = /var/log/mysql/error.log
general_log_file = /var/log/mysql/mysql.log
general_log = 1
slow_query_log_file = /var/log/mysql/slow.log

connect_timeout = 10
max_user_connections = 5
max_connections = 151
max_connect_errors = 100
port = 1781
```

Note. This figure shows changes to the *my.cnf* configuration file on Ubuntu. It has been updated to include settings related to logging, connections, and default ports.

Figure 18

```
Frank@BaldwinCountyHealth:/etc/mysql$ sudo ufw allow 1781
Rule added
Rule added (v6)
Frank@BaldwinCountyHealth:/etc/mysql$
```

Note. This figure shows that the firewall on the Baldwin County Health machine allows port 1781 for the MySQL Service.

Stored Procedures

The stored procedures for this project are designed to help the database remain secure. These procedures should be stored and ran on first initialization of the database, and as they deal with the security of the database, Adin is responsible for making and running said procedures. Figure 19 shows the first stored procedure, which removes anonymous users from the database by

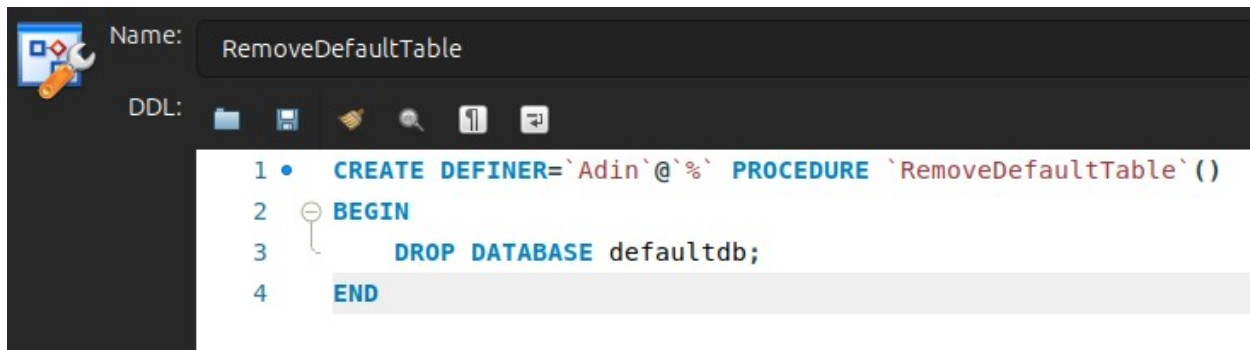
dropping all users who don't have a username. Figure 20 shows the second stored procedure drops the default database that MySQL can be configured to install named “*defaultdb*”. Figure 21 shows the procedure that sets persistent password policy features, currently enforcing a password expiration date and password history. These security settings coincide with the password policies on the machine itself as well. Figure 22 shows the final procedure which will remove the “*CREATE USER*” privilege from specified accounts which will ensure that only administrative accounts will be able to make new users on the MySQL instance. Finally, figure 23 shows that the stored procedures mentioned above are executed without error—as shown in the bottom left where it states that the query was completed.

Figure 19



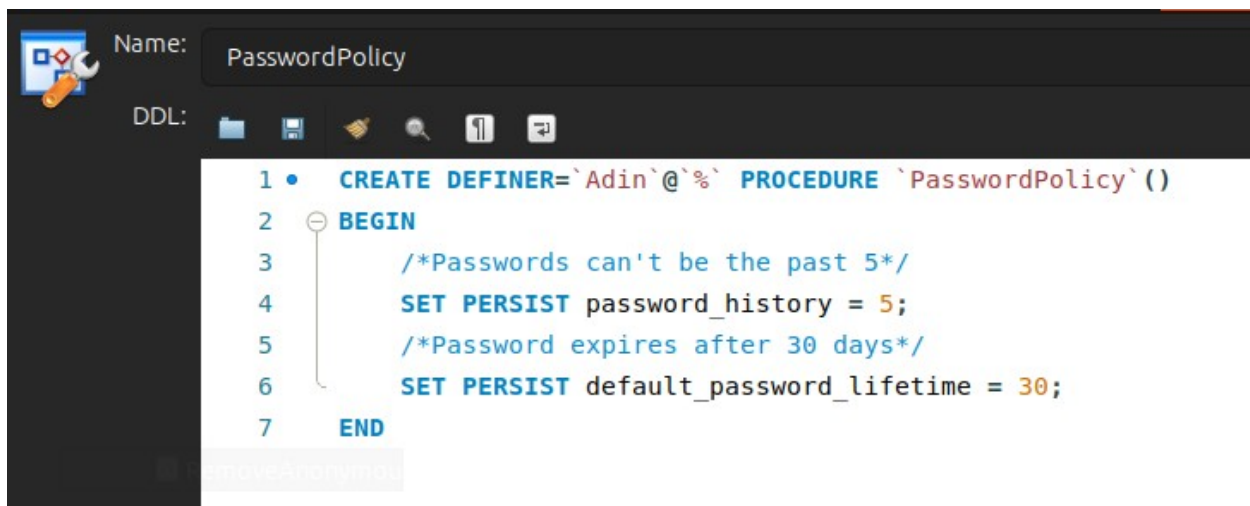
Note. This figure shows the stored procedure to remove anonymous users

Figure 20



Note. This figure shows the removal of the default database, which in Ubuntu 22.04 is "defaultdb".

Figure 21



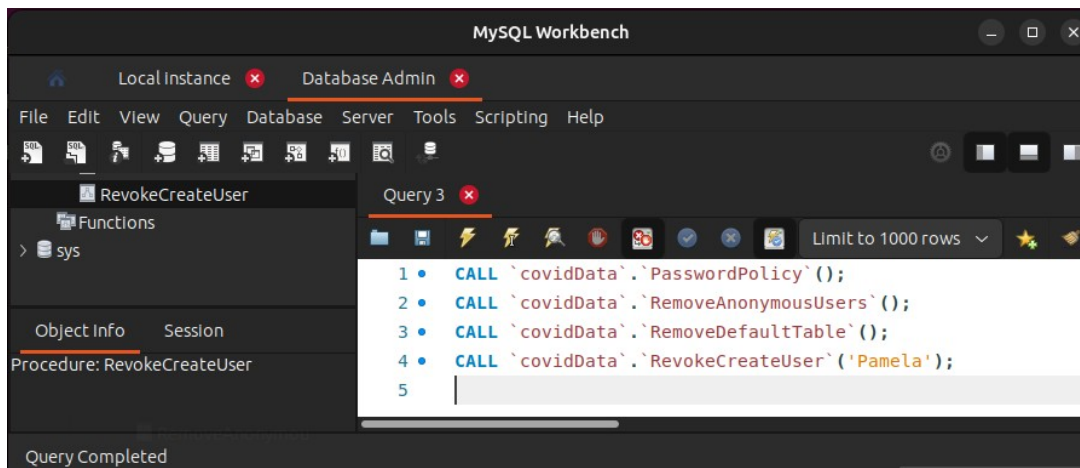
Note. This figure shows the setting of password policies. Specifically, the persistence option allows for these settings to be saved after restart.

Figure 22



Note. This figure shows the removal of the user creation privilege from all databases from specified users.

Figure 23



Note. This figure shows that all the stored procedures were executed without error from Adin's account.

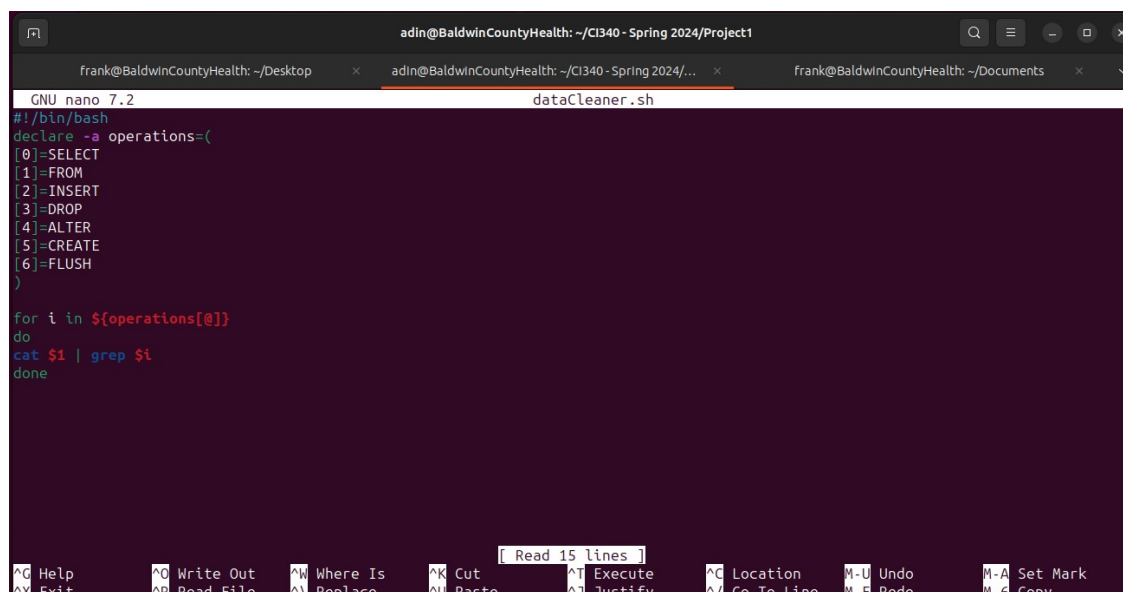
Data Loading

Sanitization

Before any data can be loaded into the MySQL instance, it first must be sanitized and checked for SQL statements, so nothing gets executed as data is loaded in. If there were SQL statements in the data that were unnoticed, that could leave security holes within the database and

surrounding environment which could potentially give attackers full access to the system. To begin data sanitization, a shell script is made to iterate through a list of SQL commands and search a given file for any results of those commands, shown in figure 24. Since the data that is expected to be loaded into the database is health-related data, there should not be anything having to do with these keywords specifically—if the data had included these statements as valid data, a new method would need to be made to sanitize the data. From running the script, shown in figure 25, the SQL statements identified are removed from the csv file before loading.

Figure 24



```

GNU nano 7.2 dataCleaner.sh
#!/bin/bash
declare -a operations=(
[0]=SELECT
[1]=FROM
[2]=INSERT
[3]=DROP
[4]=ALTER
[5]=CREATE
[6]=FLUSH
)

for i in ${operations[@]}
do
cat $1 | grep $i
done
  
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark
Exit Read File Replace Paste Justify Go To Line M-E Redo M-G Copy

Note. This figure shows the shell script under Adin’s account which will be executed to search for SQL statements in the given data before loading into the database.

Figure 25

```

adin@BaldwinCountyHealth: ~/CI340 - Spring 2024/Project1
frank@BaldwinCountyHealth: ~/Desktop x adin@BaldwinCountyHealth: ~/CI340 - Spring 2024/... x frank@BaldwinCountyHealth: ~/Documents x
adin@BaldwinCountyHealth:~/CI340 - Spring 2024/Project1$ ls
coronavirus-covid-19-pandemic-usa-counties.csv      dataCleaner.sh
'Corona-Virus Pandemic USA Counties Data Definition.txt'  PopulationEstimates.csv
adin@BaldwinCountyHealth:~/CI340 - Spring 2024/Project1$ ./dataCleaner.sh PopulationEstimates.csv
47071,TN,Hardin County,6,6,6,6,3,"26,831","26,829","ALTER USER 'root'@'localhost' IDENTIFIED BY 'NewStrongPassword!234'", "26,878"
,"27,077",-24,73,199,47,245,257,109,501,517,-62,-256,-260,0,-1,-2,38,329,452,38,328,450,0,1,9,483,482,480,482.0,9.1,9.5,18.7,19.2
,-9.5,-9.6,0.0,-0.1,12.3,16.8,12.2,16.7
48035,TX,"CREATE USER 'rogerrabbit'@'% ' IDENTIFIED BY 'password';",6,6,6,4,1,"18,235","18,226","18,245","18,495","18,697",19,250,
202,63,183,188,75,343,332,-12,-160,-144,0,8,16,30,421,321,30,429,337,1,-19,9,251,251,240,242.0,10.0,10.1,18.7,17.9,-8.7,-7.7,0.4,
0.9,22.9,17.3,23.4,18.1
48043,TX,"FLUSH PRIVILEGES;",7,7,11,11,4,"9,546","9,545","9,553","9,459","9,343",8,-94,-116,12,78,81,21,120,112,-9,-42,-31,0,20,5
9,21,-74,-147,21,-54,-88,-4,2,3,505,499,497,494.0,8.2,8.6,12.6,11.9,-4.4,-3.3,2.1,6.3,-7.8,-15.6,-5.7,-9.4
adin@BaldwinCountyHealth:~/CI340 - Spring 2024/Project1$ ./dataCleaner.sh coronavirus-covid-19-pandemic-usa-counties.csv
adin@BaldwinCountyHealth:~/CI340 - Spring 2024/Project1$

```

Note. This figure shows the output of running the script in figure 24, showing that there is SQL statements in the PopulationEstimates.csv

Loading

After sanitization of the data, the csv files can be loaded into the database using the MySQL table import wizard. Notably, the county covid data csv file is so large that the latter half of it is corrupted, and it is too big to import or even view in other programs like Excel. While limiting it to only the first 1000 rows, this still should be enough data for Pamela to complete her queries. However, this does lead to a suggestion for the rest of Frank's Cyber Security team. They need to check and verify these data sources again before sending them over to be loaded and analyzed by Pamela. It is also recommended that this data be unloaded from the database after Pamela does her queries, as the addition of SQL statements designed to make the database vulnerable is not acceptable and an indication of vulnerabilities in other areas. It can't be concluded exactly where these statements were added, as they could have been added during transit, when the data is at rest, or in the collection of data itself, but what's important is that there was a deliberate

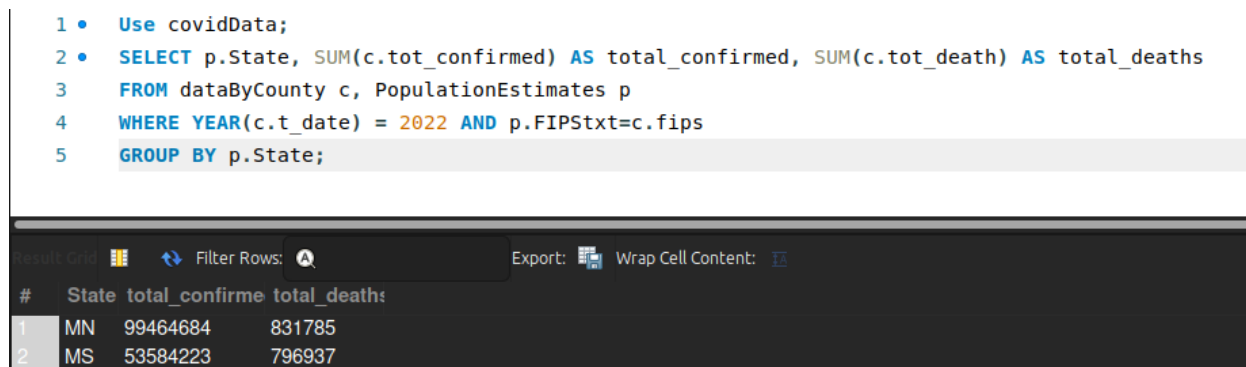
attempt to attack this database system, and as such, none of this data should be loaded. As Pamela desperately needs to do her five queries, the sanitized and partial data will be temporarily loaded.

Log Analysis

Queries

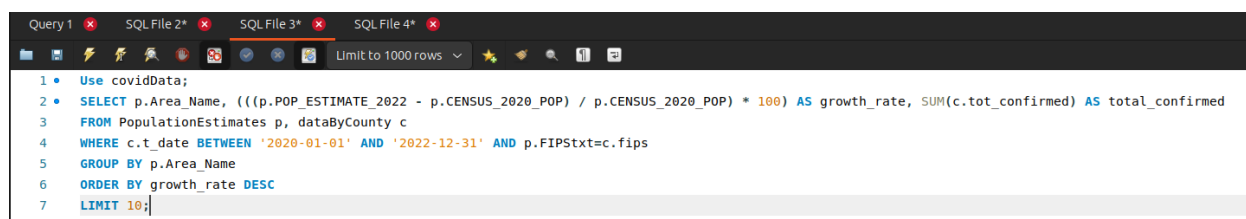
The queries that Pamela needed to run on the imported data are shown in the following figures, 26 through 30. They show the number of confirmed covid cases and deaths, the states with the highest population growth rate, domestic migration rates, economic topology data, and international migration rate compared to Covid data respectively. While each of her five queries were executed without error, only the first one gave results from the imported data—and that was with major changes to the given query. As there are numerous problems with the data regarding injected statements, lost sections, and other rampant errors associated with it, the lack of results from the queries are no surprise. The data integrity and availability have been compromised, so only one of five queries fetched data correctly.

Figure 26



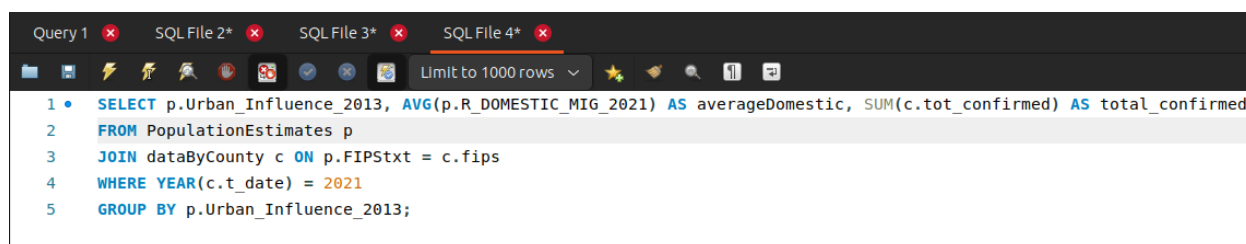
Note. This figure shows the first query, each state's total number of confirmed Covid-19 cases and deaths in 2022.

Figure 27



Note. This figure shows the second query, which counties had the highest population growth rate between 2020 and 2022, and their total number of confirmed Covid-19 cases during that period.

Figure 28



Note. This figure shows the third query, how did the domestic migration rate correlate with the total number of confirmed Covid-19 cases for each urban influence category in 2021.

Figure 29

```

1 • use covidData;
2 • SELECT p.Economic_typology_2015, SUM(c.tot_death) / SUM(p.POP_ESTIMATE_2020) * 100000 AS death_rate
3 FROM PopulationEstimates p
4 JOIN dataByCounty c ON p.FIPStxt = c.fips
5 WHERE YEAR(c.t_date) = 2020
6 GROUP BY p.Economic_typology_2015
7 ORDER BY death_rate DESC
8 LIMIT 1;
9

```

Note. This figure shows the fourth query, showing which economic typology had the highest Covid-19 death rate.

Figure 30

```

1 • Use covidData;
2 • SELECT p.Area_Name, p.R_INTERNATIONAL_MIG_2020, p.R_INTERNATIONAL_MIG_2021, p.R_INTERNATIONAL_MIG_2022, SUM(c.tot_confirmed) AS total_confirmed
3 FROM PopulationEstimates p
4 JOIN dataByCounty c ON p.FIPStxt = c.fips
5 GROUP BY p.Area_Name
6 ORDER BY total_confirmed DESC
7 LIMIT 5;

```

Note. This figure shows the fifth query, showing the international migration rate change.

Log Analysis

Following Pamela's queries, an analysis of the general log files needs to be conducted. As shown in figure 31, Pamela's queries are logged with the time, date, process ID, and user ID. In the MySQL workbench user configuration, the user with user ID 44 is Pamela, so these logged items are completely normal and show that the log settings and variables set earlier work as intended. Additionally, in figure 32, the user ID of 23—the database administrator—was the one who created the table data to load in the MySQL instance. Again, this shows that the log is working as intended and logging important events like CREATE, and queries.

Unlike Microsoft SQL, MySQL does not have a built-in log viewer/auditor, so to view and go through the logs, it must be done manually by whoever has access to it. As shown in figure 33, only the MySQL user has access to write to the log file, with users associated with the “adm” group able to read it. The adm group is the group that MySQL uses to associate with the logging administrator, which Frank has been assigned to. As there is no built-in tool for viewing and automating these logs, they need to be checked on a timely basis. Given the importance of this data and the health department, these logs should be checked every week by the logging administrator. When doing the weekly checks on the logs, Frank will need to be on the lookout for abnormal activity. As there is no feature to automatically go over the logs, there is not a way to easily make a constantly updating baseline, making it harder to identify items that are outside the baseline.

As automatic detection would not be possible in this scenario, Frank and other logging administrators will need to be extra vigilant to logged events that would be considered a high priority. As it takes time and effort to check the logs each week, the effort needs to be put into items with the highest priority and the highest risk to the environment. These kinds of items include CREATE queries, DROP queries, FLUSH queries, and ALTER queries. These queries—if executed successfully—could allow an attacker or insider to not only damage the potentially classified data but destroy the health department’s business continuity. The health department would need lots of time to repair, fix and mitigate the threat, which as a health department can be potentially fatal to the people who won’t get care in that period. Additionally, the logging administrators need to be on the lookout for numerous events in the MySQL error log. If the error log shows that are numerous login failures, that could show that there is someone with physical access to a machine on the network that doesn’t know the password. If the error log also

shows numerous errors for query-related statements, this could indicate that there is a problem with the data itself—in which case, all actions on the data need to be stopped so the data can be examined and reconstructed as soon as possible to not hinder business continuity.

If items are found in the general log and error log that are suspicious or show a potential threat to the network environment, the following actions need to be taken. The databases and machines connected to the network need to be isolated to stop the spread of infected or compromised systems. After the databases and machines are isolated, the Baldwin County Health Department must conduct a formal audit on the machines, user access control, database security, and data within the database. Notably, the data within the database will need to be checked, sanitized, and reconstructed. The data was originally found with malicious and missing data within, so it must be assumed that it might be compromised again. This formal audit will ensure that security policies and procedures are followed and not ignored. This is also the starting point for incident response to handle the potential breach of data. During this audit user permissions, local access control, physical security measures, data encryption standards, and potential backup data need to be checked. Ensuring that each account only has the specified access rights will ensure that the principle of least privilege is followed to give the least amount of access to the data. Finally, the accounts themselves need to be checked for any additional accounts on the database and machine. One of the malicious statements initially found in the data was a statement to create a user called “roggerrabbit”, so it must be assumed that this attempt may be tried again for future attacks against the server.

Figure 31


```

2024-04-16T03:45:31.773197Z 44 Query use covidData
2024-04-16T03:45:31.773386Z 44 Query SELECT DATABASE()
2024-04-16T03:45:31.774050Z 44 Query SELECT p.Economic_typology_2015, SUM(c.tot_death) / SUM(p.POP_ESTIMATE_2020) * 100000 AS death_rate
2024-04-16T03:46:57.254642Z 21 Query SHOW GLOBAL STATUS
2024-04-16T03:47:24.376175Z 44 Query Use covidData
2024-04-16T03:47:24.376446Z 44 Query SELECT DATABASE()
2024-04-16T03:47:24.377075Z 44 Query SELECT p.Area_Name, p.R_INTERNATIONAL_MIG_2020, p.R_INTERNATIONAL_MIG_2021, p.R_INTERNATIONAL_MIG_2022,
onfirmed
2024-04-16T03:47:24.941600Z 44 Query Use covidData
2024-04-16T03:47:24.941941Z 44 Query SELECT DATABASE()
2024-04-16T03:47:24.942585Z 44 Query SELECT p.Area_Name, p.R_INTERNATIONAL_MIG_2020, p.R_INTERNATIONAL_MIG_2021, p.R_INTERNATIONAL_MIG_2022,
onfirmed

```

Note. This figure shows a section of the general log file located in /var/log/mysql, specifically specifying the user ID of 44.

Figure 32

```

frank@BaldwinCountyHealth:~/Desktop$ cat /var/log/mysql/mysql.log | grep CREATE
2024-04-15T16:49:48.418950Z 23 Query CREATE TABLE `covidData`.`countyData` (`fips` text,
eath` int, `tot_confirmed` int, `location1` text, `location2` text)
2024-04-15T18:30:00.240880Z 23 Query CREATE TABLE `covidData`.`PopulationEstimates` (`FI
t, `Rural_Urban_Continuum_Code_2013` text, `Urban_Influence_2003` text, `Urban_Influence_2013` text
POP_ESTIMATE_2020` int, `POP_ESTIMATE_2021` int, `POP_ESTIMATE_2022` int, `N_POP_CHG_2020` int, `N_
HS_2022` int, `DEATHS_2020` int, `DEATHS_2021` int, `DEATHS_2022` int, `NATURAL_CHG_2020` int, `NAT
ONAL_MIG_2021` int, `INTERNATIONAL_MIG_2022` int, `DOMESTIC_MIG_2020` int, `DOMESTIC_MIG_2021` int,
`DOMESTIC_MIG_2022` int, `POP_ESTIMATE_2020` int, `POP_ESTIMATE_2021` int, `POP_ESTIMATE_2022` int, `N_POP_CHG_2020` int, `N_POP_CHG_2021` int, `N_POP_CHG_2022` int, `DEATHS_2020` int, `DEATHS_2021` int, `DEATHS_2022` int, `NATURAL_CHG_2020` int, `NATURAL_CHG_2021` int, `NATURAL_CHG_2022` int, `INTERNATIONAL_MIG_2020` int, `INTERNATIONAL_MIG_2021` int, `INTERNATIONAL_MIG_2022` int, `DOMESTIC_MIG_2020` int, `DOMESTIC_MIG_2021` int, `DOMESTIC_MIG_2022` int)

```

Note. This figure shows an additional section of the general log file, specifically searching for uses of the CREATE statement, which as shown has only been used twice by the system administrator.

Figure 33

```

frank@BaldwinCountyHealth:~/Desktop$ cat /var/log/mysql/mysql.log | grep CREATE
frank@BaldwinCountyHealth:~/Desktop$ ls -al /var/log/mysql/mysql.log
-rw-r----- 1 mysql adm 434426139 Apr 15 20:51 /var/log/mysql/mysql.log
frank@BaldwinCountyHealth:~/Desktop$

```

Note. This figure shows the access rights of the MySQL general log file.

References

Canonical. (2024). *Snapcraft Store*. Canonical Snapcraft. <https://snapcraft.io/search?q=mysql>

Ellingwood, Justin. (2022). *How to use `GRANT` and `REVOKE` to manage privileges in MySQL*. Prisma. <https://www.prisma.io/dataguide/mysql/authentication-and-authorization/privilege-management>

Oracle. (2024). *MySQL 8.0 Reference Manual: Revoke*. MySQL.
<https://dev.mysql.com/doc/refman/8.0/en/revoke.html>

Oracle. (2024). *MySQL 8.3 Reference Manual: Privileges*. MySQL.
<https://dev.mysql.com/doc/refman/8.3/en/privileges-provided.html>

Reid, Phillip. (2024). *Defense In Depth*. Database Security [Power Point].

Reid, Phillip. (2024). *Database Security Auditing*. Database Security [Power Point].

Reid, Phillip. (2024). *Securing MySQL*. Database Security [Power Point].

Reid, Phillip. (2024). *MySQL Installation & Configuration*. Database Security [Power Point].